

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
Пермский национальный исследовательский политехнический университет
Электротехнический факультет
Кафедра информационных технологий и автоматизированных систем

ОТЧЕТ
о лабораторной работе 2
Семестр: 4

На тему: «Алгоритм Прима»

Выполнил студент ИВТ-24-26:

Шишкин М.Г

(дата, подпись)

Проверил:

Рустамханова Г.И

(дата, подпись)

Пермь 2026

Содержание

1 Постановка задачи	3
2 Краткое описание решения	4
3 Код	5
4 Тестирование программы	11
5 Вывод	12
6 Github	13

1 Постановка задачи

Требуется реализовать алгоритм Прима для построения минимального островного дерева взвешенного неориентированного графа. На вход подается матрица смежности размером 10×10 , содержащая целые неотрицательные числа, где 0 обозначает отсутствие ребра, а положительное число - вес ребра. Граф является неориентированным, поэтому матрица должна быть симметричной относительно главной диагонали, на которой расположены нули. Программа должна читать матрицу из файла, выполнять проверку корректности входных данных, строить минимальное островное дерево, начиная с первой вершины, и выводить перечень ребер, вошедших в островное дерево, с указанием веса каждого ребра, а также итоговый суммарный вес дерева.

2 Краткое описание решения

Решение основано на классическом алгоритме Прима для поиска минимального остовного дерева. Основная идея: дерево строится последовательным добавлением вершин. Начальная вершина выбирается первой (вершина 0). На каждом шаге из множества еще не добавленных вершин выбирается та, которая соединена с уже построенной частью дерева ребром минимального веса. Это ребро добавляется в остовное дерево, а вершина помечается как посещенная. Процесс повторяется до тех пор, пока все вершины не будут включены в дерево. Для хранения минимального веса ребра к каждой вершине используется массив `minEdge`, для хранения родительской вершины — массив `parent`, для отметки включенных вершин — массив `selected`. Перед выполнением алгоритма производится проверка входной матрицы: на отсутствие отрицательных чисел, симметричность, нули на диагонали и связность графа с помощью обхода в глубину (DFS).

3 Код

```
1  #include <iostream>
2  #include <vector>
3  #include <climits>
4  #include <fstream>
5  #include <string>
6
7  using namespace std;
8
9  class Primalgorithm {
10 private:
11     static const int SIZE = 10;
12     int graph[SIZE][SIZE];
13     bool selected[SIZE];
14     int parent[SIZE];
15     int minEdge[SIZE];
16
17     // Проверка матрицы
18     bool validateMatrix() {
19         for (int i = 0; i < SIZE; i++) {
20             for (int j = 0; j < SIZE; j++) {
21                 // Проверка на отрицательные числа
22                 if (graph[i][j] < 0) {
23                     cout << "Ошибка: матрица содержит отрицательное число [" << i << "][" << j << "] = " << graph[i][j] << endl;
24                     return false;
25                 }
26
27                 // Проверка на несимметричность
28                 if (graph[i][j] != graph[j][i]) {
29                     cout << "Ошибка: матрица несимметрична [" << i << "][" << j << "] = " << graph[i][j]
30                     << ", [" << j << "][" << i << "] = " << graph[j][i] << endl;
31                     return false;
32                 }
33
34                 // Проверка главной диагонали
35                 if (i == j && graph[i][j] != 0) {
36                     cout << "Ошибка: на главной диагонали должен быть 0, а найдено [" << i << "][" << i << "] = " << graph[i][j] << endl;
37                     return false;
38                 }
39             }
40         }
41         return true;
42     }
43
44     // Проверка связности графа
45     bool isGraphConnected() {
46         vector<bool> visited(SIZE, false);
47
48         // Поиск первой вершины с ненулевыми ребрами
49         int startVertex = -1;
50         for (int i = 0; i < SIZE; i++) {
```

```

49     int startVertex = -1;
50     for (int i = 0; i < SIZE; i++) {
51         for (int j = 0; j < SIZE; j++) {
52             if (graph[i][j] > 0) {
53                 startVertex = i;
54                 break;
55             }
56         }
57         if (startVertex != -1) break;
58     }
59
60     if (startVertex == -1) {
61         cout << "Ошибка: граф не содержит ребер" << endl;
62         return false;
63     }
64
65     // DFS для проверки связности
66     dfs(startVertex, visited);
67
68     // Проверка, все ли вершины с ребрами посещены
69     for (int i = 0; i < SIZE; i++) {
70         bool hasEdges = false;
71         for (int j = 0; j < SIZE; j++) {
72             if (graph[i][j] > 0) {
73                 hasEdges = true;
74                 break;
75             }
76         }
77         if (hasEdges && !visited[i]) {
78             cout << "Ошибка: граф несвязный" << endl;
79             return false;
80         }
81     }
82
83     return true;
84 }
85
86 void dfs(int vertex, vector<bool>& visited) {
87     visited[vertex] = true;
88     for (int i = 0; i < SIZE; i++) {
89         if (graph[vertex][i] > 0 && !visited[i]) {
90             dfs(i, visited);
91         }
92     }
93 }
94
95 public:
96     PrimAlgorithm() {
97         // Инициализация
98         for (int i = 0; i < SIZE; i++) {

```

```

97 // Инициализация
98 for (int i = 0; i < SIZE; i++) {
99     for (int j = 0; j < SIZE; j++) {
100         graph[i][j] = 0;
101     }
102     selected[i] = false;
103     parent[i] = -1;
104     minEdge[i] = INT_MAX;
105 }
106
107
108 // Чтение матрицы из файла
109 bool readGraphFromFile(const string& filename) {
110     ifstream file(filename);
111
112     if (!file.is_open()) {
113         cout << "Ошибка: не удалось открыть файл " << filename << endl;
114         return false;
115     }
116
117     cout << "Чтение матрицы из файла " << filename << "... " << endl;
118
119     for (int i = 0; i < SIZE; i++) {
120         for (int j = 0; j < SIZE; j++) {
121             if (!(file >> graph[i][j])) {
122                 cout << "Ошибка: недостаточно данных в файле" << endl;
123                 file.close();
124                 return false;
125             }
126         }
127     }
128
129     file.close();
130
131     // Проверка на корректность данных
132     if (!validateMatrix()) {
133         return false;
134     }
135
136     if (!isGraphConnected()) {
137         return false;
138     }
139
140     cout << "Матрица успешно загружена и прошла все проверки!" << endl;
141     return true;
142 }
143
144 // Вывод матрицы
145 void printGraph() {
146     cout << "\nЗагруженная матрица смежности:" << endl;

```

```

145 void printGraph() {
146     cout << "\nЗагруженная матрица смежности:" << endl;
147     for (int i = 0; i < SIZE; i++) {
148         for (int j = 0; j < SIZE; j++) {
149             cout << graph[i][j] << "\t";
150         }
151         cout << endl;
152     }
153 }
154
155 // Алгоритм Прима
156 void buildMST() {
157     // Сброс массивов
158     for (int i = 0; i < SIZE; i++) {
159         selected[i] = false;
160         parent[i] = -1;
161         minEdge[i] = INT_MAX;
162     }
163
164     minEdge[0] = 0;
165
166     cout << "\n=== Построение минимального остовного дерева (алгоритм Прима) ===" << endl;
167
168     for (int count = 0; count < SIZE; count++) {
169         // Выбираем вершину с минимальным ребром
170         int v = -1;
171         for (int i = 0; i < SIZE; i++) {
172             if (!selected[i] && (v == -1 || minEdge[i] < minEdge[v])) {
173                 v = i;
174             }
175         }
176
177         if (v != -1) {
178             selected[v] = true;
179
180             if (parent[v] != -1) {
181                 cout << "Добавлено ребро: " << parent[v] + 1 << " -- " << v + 1
182                     << " вес: " << graph[parent[v]][v] << endl;
183             }
184
185             // Обновляем минимальные ребра
186             for (int to = 0; to < SIZE; to++) {
187                 if (graph[v][to] > 0 && !selected[to] && graph[v][to] < minEdge[to]) {
188                     minEdge[to] = graph[v][to];
189                     parent[to] = v;
190                 }
191             }
192         }
193     }
194 }

```



```

193     }
194 }
195
196 // Вывод результата
197 void printResult() {
198     cout << "\n=== Результат работы алгоритма Прима ===" << endl;
199     cout << "Перечень ребер остоного дерева:" << endl;
200
201     int totalWeight = 0;
202     for (int i = 1; i < SIZE; i++) {
203         if (parent[i] != -1) {
204             cout << "Ребро: " << parent[i] + 1 << " - " << i + 1
205                 << ", вес: " << graph[parent[i]][i] << endl;
206             totalWeight += graph[parent[i]][i];
207         }
208     }
209
210     cout << "\nИТОГОВЫЙ ВЕС ДЕРЕВА: " << totalWeight << endl;
211 }
212 };
213
214 // Функция для создания примера матрицы в файле
215 void createExampleFile(const string& filename) {
216     ofstream file(filename);
217
218     if (file.is_open()) {
219         // Пример связанного графа
220         int exampleMatrix[10][10] = {
221             {0, 2, 0, 6, 0, 0, 0, 0, 0, 0},
222             {2, 0, 3, 8, 5, 0, 0, 0, 0, 0},
223             {0, 3, 0, 0, 7, 0, 0, 0, 0, 0},
224             {6, 8, 0, 0, 9, 0, 0, 0, 0, 0},
225             {0, 5, 7, 9, 0, 4, 0, 0, 0, 0},
226             {0, 0, 0, 0, 4, 0, 1, 0, 0, 0},
227             {0, 0, 0, 0, 0, 1, 0, 2, 0, 0},
228             {0, 0, 0, 0, 0, 0, 2, 0, 3, 0},
229             {0, 0, 0, 0, 0, 0, 0, 3, 0, 4},
230             {0, 0, 0, 0, 0, 0, 0, 0, 4, 0}
231         };
232
233         for (int i = 0; i < 10; i++) {
234             for (int j = 0; j < 10; j++) {
235                 file << exampleMatrix[i][j];
236                 if (j < 9) file << " ";
237             }
238             file << endl;
239         }
240
241         file.close();
242         cout << "Файл-пример " << filename << " создан!" << endl;

```

```

238         file << endl;
239     }
240
241     file.close();
242     cout << "Файл-пример " << filename << " создан!" << endl;
243 }
244 else {
245     cout << "Ошибка при создании файла" << endl;
246 }
247 }
248
249 int main() {
250     setlocale(LC_ALL, "Russian");
251
252     PrimAlgorithm prim;
253     string filename;
254
255     cout << "=== Лабораторная работа: Алгоритм Прима ===" << endl;
256     cout << "Минимальное остовное дерево (MST)" << endl;
257     cout << "Размер матрицы: 10x10" << endl;
258     cout << "===== " << endl;
259
260     cout << "\nВведите имя файла с матрицей (например, graph.txt): ";
261     cin >> filename;
262
263     // Пытаемся прочитать файл
264     if (!prim.readGraphFromFile(filename)) {
265         cout << "\nХотите создать пример файла для тестирования? (y/n): ";
266         char choice;
267         cin >> choice;
268
269         if (choice == 'y' || choice == 'Y') {
270             createExampleFile(filename);
271             cout << "Запустите программу снова и укажите этот файл." << endl;
272         }
273         return 1;
274     }
275
276     // Выводим матрицу
277     prim.printGraph();
278
279     // Строим остовное дерево
280     prim.buildMST();
281
282     // Выводим результат
283     prim.printResult();
284
285     return 0;
286 }

```

4 Тестирование программы

Введите имя файла с матрицей (например, graph.txt): g11.txt

Чтение матрицы из файла g11.txt...

Матрица успешно загружена и прошла все проверки!

Загруженная матрица смежности:

0	0	0	7	8	6	3	0	0	0
0	0	0	4	2	1	0	3	9	1
0	0	0	5	9	0	6	8	5	0
7	4	5	0	0	0	6	0	0	9
8	2	9	0	0	0	0	4	5	0
6	1	0	0	0	0	3	0	7	0
3	0	6	6	0	3	0	0	3	0
0	3	8	0	4	0	0	0	4	5
0	9	5	0	5	7	3	4	0	8
0	1	0	9	0	0	0	5	8	0

=== Построение минимального остовного дерева (алгоритм Прима) ===

Добавлено ребро: 1 -- 7 вес: 3

Добавлено ребро: 7 -- 6 вес: 3

Добавлено ребро: 6 -- 2 вес: 1

Добавлено ребро: 2 -- 10 вес: 1

Добавлено ребро: 2 -- 5 вес: 2

Добавлено ребро: 2 -- 8 вес: 3

Добавлено ребро: 7 -- 9 вес: 3

Добавлено ребро: 2 -- 4 вес: 4

Добавлено ребро: 9 -- 3 вес: 5

=== Результат работы алгоритма Прима ===

Перечень ребер остовного дерева:

Ребро: 6 - 2, вес: 1

Ребро: 9 - 3, вес: 5

Ребро: 2 - 4, вес: 4

Ребро: 2 - 5, вес: 2

Ребро: 7 - 6, вес: 3

Ребро: 1 - 7, вес: 3

Ребро: 2 - 8, вес: 3

Ребро: 7 - 9, вес: 3

Ребро: 2 - 10, вес: 1

ИТОГОВЫЙ ВЕС ДЕРЕВА: 25

5 Вывод

В ходе выполнения лабораторной работы был реализован алгоритм Прима для построения минимального остовного дерева. Программа успешно считывает матрицу смежности из файла, проверяет ее на соответствие требованиям (симметричность, отсутствие отрицательных чисел, нули на диагонали, связность графа), строит минимальное остовное дерево и выводит список ребер с их весами, а также суммарный вес дерева.

6 Github

<https://github.com/MAKSPOWERO/math>